

# ARTEFATO 01 — Outline Filosófico-Técnico

IA como Processo × Dinheiro como Energia × Baixo Nível de Linguagem

"Da Abstração à Instrução: o Caminho do Bit ao Sentido"

 **Status:** Artefato Intermediário #01 | Série: Formalização Progressiva  Sessão: Inaugural | Próximo: Artefato 02 → Arquitetura de Módulos

## 0. Meta-Estrutura do Documento



### LEGENDA DE ÍCONES:

-  = Fluxo / Processo
-  = Filosofema / Insight
-  = Formalismo / Engenharia
-  = Rationale / Justificativa
-  = Metacrítica
-  = Engenharia de Prompt
-  = Engenharia de Testes
-  = Requisito
-  = Usabilidade / UX
-  = Baixo Nível / Assembly / C
-  = Risco / Alerta
-  = Validado / Aprovado

### Filosofema-Raiz:

 "Um sistema vivo não é descrito — é habitado. Este documento não é um mapa, é o primeiro passo da dança."

## 1. O Problema da Camada: Da Filosofia ao Bit

### 1.1 A Torre de Babel Invertida

A maioria dos projetos de IA começa no topo (filosofia, produto) e nunca desce ao chão (hardware, instrução). Nosso projeto **desce intencionalmente** — porque é no nível baixo que os processos *realmente vivem*.



- NÍVEL 7: Filosofema ("dinheiro como energia")
- NÍVEL 6: Arquitetura de sistema (daemon, loop)
- NÍVEL 5: Linguagem de alto nível (Python, JS)
- NÍVEL 4: Runtime / VM (CPython, V8, JVM)
- NÍVEL 3: Sistema operacional (syscalls, scheduler)
- NÍVEL 2: Assembly / IR (LLVM IR, WASM)
- NÍVEL 1: Linguagem de máquina (opcodes)
- NÍVEL 0: Hardware (transistores, clock, volt)



**Rationale:** Quando dizemos "IA como processo contínuo (daemon)", estamos descrevendo comportamento que, no nível 0, é um **loop de clock** que nunca para. A metáfora filosófica e a realidade física são **isomorfas**.

## 1.2 Por que Baixo Nível Importa Aqui

| Camada           | O que ganha                              | Risco se ignorar                       |
|------------------|------------------------------------------|----------------------------------------|
| Assembly / C     | Latência previsível, controle de memória | GC pauses matam real-time              |
| WASM             | Portabilidade + near-native speed        | Lock-in de runtime                     |
| GGUF / llama.cpp | SLM local sem overhead                   | Quantização mal feita = qualidade ruim |
| Syscalls diretas | Daemon sem framework                     | Complexidade de manutenção             |



"O filósofo diz 'fluxo'. O engenheiro de sistemas diz 'non-blocking I/O'. São o mesmo rio com nomes diferentes."

## 2. Formalismos Fundamentais

### 2.1 IA como Processo: Formalismo de Estado

**Definição formal** (inspirada em máquina de estados + process calculus):



$P = (S, \Sigma, \delta, s_0, F)$   
onde:  
 $S$  = conjunto de estados (contexto, memória, heurísticas)  
 $\Sigma$  = alfabeto de eventos (preços, tweets, blocos on-chain)  
 $\delta$  = função de transição:  $S \times \Sigma \rightarrow S \times \text{Ações}$   
 $s_0$  = estado inicial (embeddings base + perfil do investidor)  
 $F = \emptyset \leftarrow$  NÃO HÁ ESTADO FINAL (processo eterno)



**Rationale:**  $F = \emptyset$  é a chave filosófica. Um produto tem  $F \neq \emptyset$  (entrega e para). Um processo tem  $F = \emptyset$  (transição perpétua). Essa distinção muda TUDO na arquitetura.

### 2.2 Dinheiro como Energia: Formalismo Termodinâmico



$E_{financeira}(t) = \int [0 \rightarrow t] (\text{fluxo\_entrada}(\tau) - \text{fluxo\_saída}(\tau)) \, dt$

onde:  
fluxo\_entrada = staking\_rewards + LP\_fees + yield  
fluxo\_saída = gas\_fees + impermanent\_loss + slippage

Entropia Financeira:  
 $H = -\sum p(i) \times \log(p(i)) \leftarrow$  diversificação de ativos

Condição de Equilíbrio Dinâmico:  
 $d(E)/dt \approx 0$  NÃO significa parado — significa dança estável

💡 "Capital em equilíbrio dinâmico não é capital morto — é capital que dança sem cair."

### 2.3 Formalismo de Loop em Baixo Nível (Pseudoassembly)



asm

; IA\_DAEMON\_LOOP — pseudoassembly para clareza conceitual  
; Registradores: RCX=contexto, RDX=evento, RAX=ação, RSI=estado

DAEMON\_START:  
CALL load\_persistent\_state ; RSI ← estado do disco  
MOV RCX, RSI ; contexto ← estado

MAIN\_LOOP:  
CALL poll\_event\_queue ; RDX ← próximo evento (non-blocking)  
TEST RDX, RDX  
JZ CHECK\_SCHEDULE ; sem evento → verifica scheduler

PUSH RCX ; salva contexto  
CALL llm\_infer ; RAX ← inferência (RDX+RCX como input)  
CALL plan\_action ; RAX ← ação planejada  
CALL execute\_via\_mcp ; executa (side-effects externos)  
CALL observe\_result ; RCX ← novo contexto  
CALL update\_memory ; persiste estado  
CALL collect\_feedback ; avalia resultado

CMP feedback\_score, THRESHOLD  
JL AUTO\_OPTIMIZE ; abaixo do limiar → otimiza

JMP MAIN\_LOOP ; eterno

CHECK\_SCHEDULE:  
CALL check\_cron ; há tarefa agendada?  
JNZ MAIN\_LOOP  
CALL sleep\_microseconds ; yield para OS  
JMP MAIN\_LOOP

AUTO\_OPTIMIZE:  
CALL adjust\_heuristics ; calibração fina  
CALL swap\_lora\_adapter ; troca LoRA se necessário  
JMP MAIN\_LOOP

; ← NUNCA HÁ RET AQUI. F = ∅.

 **Rationale:** Escrever em pseudoassembly força a pensar em **o que realmente acontece**: `CALL sleep_microseconds` é o "aguardar(3600)" do documento original — mas no nível real, é um `nanosleep()` ou `epoll_wait()` que **cede o CPU** sem parar o processo.

---

## 3. Engenharia de Prompt

### 3.1 Princípios para o Sistema Descrito



PROMPT ENGINEERING PRINCIPLES para IA-como-Processo:

- [P1] PROMPT DE SISTEMA (imutável, carregado 1x):
  - Define personalidade, limites éticos, domínio
  - Ex: "Você é um gestor de portfolio DeFi conservador..."
- [P2] PROMPT DE CONTEXTO (atualizado a cada ciclo):
  - Injeta estado atual: preços, alertas, histórico recente
  - Gerado dinamicamente via RAG
- [P3] PROMPT DE TAREFA (evento-driven):
  - Específico da ação corrente
  - Ex: "Analise este pico de mindshare em JEWEL..."
- [P4] PROMPT DE META (auto-otimização):
  - "Avalie sua resposta anterior. O trade sugerido resultou em +2.3%. Ajuste sua heurística de risco."

### 3.2 Template de Prompt Adaptativo (para RAG Engine)



xml

```
<prompt_template versao="1.0">
<sistema>
  Você é um agente de investimento Web3 operando como processo contínuo.
  Perfil do usuário: {perfil_risco}
  Capital disponível: {capital_atual}
  Histórico de acertos: {taxa_acerto_30d}%
</sistema>

<contexto>
  Timestamp: {unix_timestamp}
  Mercado: {resumo_mercado_24h}
  Portfolio atual: {snapshot_portfolio}
  Eventos recentes: {ultimos_n_eventos}
  LoRA ativa: {lora_especializada}
</contexto>

<tarefa>
  Evento detectado: {tipo_evento}
  Dados: {payload_evento}
  Pergunta: {questao_especifica}
</tarefa>

<restricoes>
  Max exposição por token: {limite_exposicao}%
  Requer confirmação humana se: valor > {limite_autonomia}
  Explique em linguagem simples ao final.
</restricoes>
</prompt_template>
```

### 3.3 Engenharia de Contexto (Context Engineering)



JANELA DE CONTEXTO = recurso escasso → gestão crítica

Estratégia de Compressão:

|              |                        |              |
|--------------|------------------------|--------------|
| [PERMANENTE] | Sistema + Regras       | ~500 tokens  |
| [DESLIZANTE] | Últimos N eventos      | ~1000 tokens |
| [COMPRIMIDO] | Resumo histórico (RAG) | ~500 tokens  |
| [DINÂMICO]   | Tarefa atual           | ~500 tokens  |

---

TOTAL GERENCIADO: ~2500 tokens  
(deixa margem para resposta + chain-of-thought)

Técnica de Sliding Window com Summarization:  
a cada K ciclos:  
sumário = llm\_compress(histórico[-K:])  
histórico = [sumário] + histórico[-3:] ← mantém recência

 **Metacrítica:** Context engineering é onde 90% dos projetos falham. Não é "jogar tudo no contexto" — é **editar** como um escritor edita um roteiro.

---

## 4. Engenharia de Requisitos

### 4.1 Requisitos Funcionais (RF)



- RF-01: O sistema DEVE operar como daemon contínuo (uptime > 99.5%)
- RF-02: O sistema DEVE detectar padrões de hype com precisão > 70%
- RF-03: O sistema DEVE executar análise de sentimento em < 5 segundos
- RF-04: O sistema DEVE manter memória persistente entre sessões
- RF-05: O sistema DEVE gerar prompts adaptativos via RAG
- RF-06: O sistema DEVE suportar perfis de risco: conservador / moderado / arrojado
- RF-07: O sistema DEVE integrar com MCP para execução de ações externas
- RF-08: O sistema DEVE exigir confirmação humana para ordens > R\$50,00
- RF-09: O sistema DEVE auto-ajustar limiares com base em acurácia histórica
- RF-10: O sistema DEVE rodar SLM local (Phi-3 ou equivalente) sem dependência de nuvem

### 4.2 Requisitos Não-Funcionais (RNF)



- RNF-01: PERFORMANCE → latência de inferência local < 2s (hardware médio)
- RNF-02: SEGURANÇA → chaves privadas NUNCA no contexto do LLM
- RNF-03: PRIVACIDADE → dados do usuário processados localmente (edge-first)
- RNF-04: OBSERVABILIDADE → logs estruturados (JSON) de todos os ciclos
- RNF-05: RESILIÊNCIA → crash recovery em < 30s (estado persistido)
- RNF-06: PORTABILIDADE → roda em Raspberry Pi 5 / MacBook Air / Linux x86
- RNF-07: AUDITABILIDADE → histórico imutável de decisões (append-only log)

### 4.3 Requisitos de Baixo Nível (RBN) — Camada C/Sistema



- RBN-01: Daemon implementado com fork() + setsid() OU systemd unit
- RBN-02: I/O não-bloqueante via epoll/kqueue para zero-latency event handling
- RBN-03: Modelo GGUF carregado via mmap() para evitar cópias desnecessárias
- RBN-04: Contexto serializado em formato binário compacto (MessagePack > JSON)
- RBN-05: Watchdog timer (SIGALRM) para detectar travamentos de inferência
- RBN-06: Memory-mapped state file para persistência instantânea (msync)
- RBN-07: Uso de huge pages se disponível (TLB optimization para matrizes)

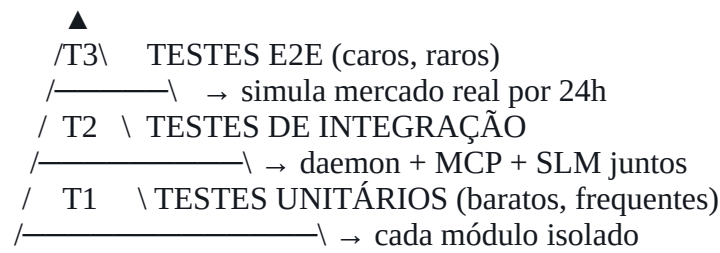
 **Rationale dos RBN:** O RNF-01 exige < 2s de latência. Sem os RBN, um modelo 3B quantizado em hardware modesto facilmente toma 5-8s. Os RBN são o que transforma a promessa filosófica em realidade física.

---



## 5. Engenharia de Testes

### 5.1 Pirâmide de Testes para o Sistema



### 5.2 Casos de Teste Críticos



- [T1-01] UNIT: Detector de Hype  
ENTRADA: mindshare\_cresc=0.35, preco\_cresc=0.55  
ESPERADO: alerta=TRUE  
CASO NEGATIVO: mindshare\_cresc=0.10, preco\_cresc=0.20 → alerta=FALSE
- [T1-02] UNIT: Compressão de Contexto  
ENTRADA: histórico de 50 eventos  
ESPERADO: sumário ≤ 500 tokens sem perda de eventos críticos
- [T2-01] INTEGRAÇÃO: Ciclo Completo  
CENÁRIO: evento de preço simulado injetado via MCP mock  
ESPERADO: decisão gerada + log estruturado em < 5s
- [T2-02] INTEGRAÇÃO: Crash Recovery  
CENÁRIO: kill -9 no processo durante inferência  
ESPERADO: reinício em < 30s com estado restaurado
- [T3-01] E2E: Backtesting 30 dias  
DADOS: histórico real de ONE/JEWEL (Harmony)  
MÉTRICAS: acurácia de alertas, P&L simulado, drawdown máximo
- [T3-02] E2E: Autonomia 24h  
CENÁRIO: mercado normal + 1 evento de hype simulado  
ESPERADO: sistema opera sem intervenção, alertas corretos, confirmação solicitada para ordem > limiar

### 5.3 Teste de Regressão de Prompt (Prompt Testing)



python

# Conceito: cada mudança de prompt deve passar em suite de testes

```
CASOS_DE_TESTE_PROMPT = [
{
  "id": "PT-001",
  "cenario": "hype claro",
  "contexto": {...dados_de_bolha...},
  "resposta_esperada_contém": ["risco", "reduzir", "exposição"],
  "resposta_nao_deve_conter": ["comprar mais", "oportunidade"],
},
{
  "id": "PT-002",
  "cenario": "mercado estável",
  "contexto": {...dados_normais...},
  "resposta_esperada_contém": ["manter", "monitorar"],
}
]
```

```
def testar_prompt(template, casos):
  for caso in casos:
    resposta = llm_infer(template.format(**caso["contexto"]))
    assert all(k in resposta for k in caso["resposta_esperada_contém"])
    assert not any(k in resposta for k in caso.get("resposta_nao_deve_conter", []))
```

## 👉 6. Engenharia de Usabilidade

### 6.1 Princípio da Complexidade Oculta

📄

✓

VISÍVEL PARA O USUÁRIO (camada de cristal):

|    |                                      |  |
|----|--------------------------------------|--|
| 📊  | Sua energia financeira: R\$1.247 ↑3% |  |
| ⚠️ | Risco detectado: JEWEL (87%)         |  |
| 💡  | Sugestão: reduzir GameFi para 15%    |  |
| ✅  | Hedge executado: 5% do portfolio     |  |

INVISÍVEL (camada de lava):

daemon loop → RAG → SLM → MCP → epoll →  
mmap → quantização → embeddings → limiares →  
auto-otimização → LoRA → syscalls → clock

💡 "A boa UX é como a boa saúde: só percebida quando falta."

### 6.2 Princípios UX para o Caso R\$300

📄

✓



- [UX-01] CONFIANÇA PRIMEIRO
- Toda decisão autônoma deve ser explicável em 1 frase simples
  - Ex: "Vendi 10% de JEWEL porque o rumor cresceu 40% sem movimento real de preço. Isso costuma preceder queda."

- [UX-02] GRADIENTE DE AUTONOMIA
- Nível 1: só notificações (usuário decide tudo)
  - Nível 2: sugestões com 1-clique para executar
  - Nível 3: execução automática abaixo de limiar
  - Nível 4: full auto (apenas para usuários avançados)
  - DEFAULT para R\$300: Nível 2

- [UX-03] LINGUAGEM DE ENERGIA (não de finanças)
- NÃO: "Seu portfolio sofreu drawdown de 3.2%"
  - SIM: "Sua energia financeira perdeu um pouco de calor hoje. Nada que um ciclo de staking não recupere."

- [UX-04] FEEDBACK EM LOOP VISÍVEL
- Mostrar ao usuário quando o sistema aprendeu algo
  - Ex: "Com base nos últimos 15 alertas, ajustei meu sensor de hype. Agora sou 8% mais preciso."

6.3 Jornada do Usuário (R\$300, Perfil Conservador)



DIA 1: Cadastro → define perfil → conecta wallet  
DIA 2: Primeiro relatório de energia financeira  
DIA 3-7: Notificações passivas, sem ação autônoma  
SEMANA 2: Primeiro alerta de hype → usuário decide  
SEMANA 3: Sistema sugere primeira ação → usuário aprova  
MÊS 2: Usuário confia → habilita Nível 3 (semi-auto)  
MÊS 6: Sistema tem histórico real → acurácia > 75%

7. Metacríticas e Riscos

7.1 Metacríticas do Projeto

| #     | Metacrítica                                                | Gravidade | Mitigação                                                |
|-------|------------------------------------------------------------|-----------|----------------------------------------------------------|
| MC-01 | Filosofemas bonitos ≠ código funcional                     | Alta      | Cada filosofema deve ter um RBN correspondente           |
| MC-02 | SLM local pode ser inadequado para análise financeira      | Média     | Fine-tuning com dados DeFi + avaliação contínua          |
| MC-03 | Auto-otimização pode reforçar vieses                       | Alta      | Revisão humana quinzenal + versioning de heurísticas     |
| MC-04 | R\$300 é capital pequeno → fees podem devorar retorno      | Alta      | Modelo de simulação antes de executar ordens reais       |
| MC-05 | MCP ainda é protocolo jovem → breaking changes             | Média     | Abstrair com adapter pattern                             |
| MC-06 | Privacidade de dados locais vs. qualidade de modelos cloud | Baixa     | Opção híbrida: análise local, execução cloud com consent |

## 7.2 O Paradoxo do Processo Auto-Referente

 **Metacrítica Filosófica Profunda:** Um sistema que se auto-otimiza baseado em seus próprios resultados pode entrar em loop fechado de reforço. É o equivalente computacional da câmara de eco.

**Solução Proposta:** Injetar periodicamente *ruído controlado* — dados de mercados não relacionados, perspectivas contrárias — como um **antiviés ativo**. A diversidade de input é a imunidade do processo.

## 8. Próximos Artefatos (Roadmap da Série)



- ARTEFATO 01 (este) ← VOCÊ ESTÁ AQUI
  - Outline filosófico-técnico, requisitos, testes, UX
- ARTEFATO 02 → Arquitetura de Módulos
  - Diagrama detalhado de componentes, interfaces, contratos
- ARTEFATO 03 → Daemon em C / Python híbrido
  - Código real do loop principal com epoll e mmap
- ARTEFATO 04 → RAG Engine + Prompt Generator
  - Implementação do sistema de prompts adaptativos
- ARTEFATO 05 → Integração MCP
  - Servidor MCP para execução de ordens
- ARTEFATO 06 → LoRA Fine-tuning para DeFi
  - Dataset, treinamento, avaliação
- ARTEFATO 07 → Dashboard de Energia Financeira
  - UI/UX com linguagem de processo, não de estoque
- ARTEFATO 08 → Backtesting com ONE/JEWEL
  - Validação real com histórico de preços
- ARTEFATO 09 → Deploy: R\$300 Real
  - Guia de produção para o caso descrito no anexo

## 9. Filosofemas Finais deste Artefato

-  "Assembly não é o oposto de filosofia. É onde a filosofia toca o real."
-  " $F = \emptyset$  não é ausência de fim — é presença eterna de começo."
-  "O bug mais caro não está no código. Está na metáfora errada escolhida no início."
-  "Um daemon que dorme não é um daemon morto — é um daemon que sonha com o próximo evento."
-  "Documentar é a primeira forma de testar. Se não consegue explicar, não consegue implementar."

# Convenção de Continuidade

Para manter coerência entre artefatos ao longo da conversa:



TODA SESSÃO COMEÇA COM:

- Revisão do último artefato
- Atualização de status dos requisitos
- Identificação do próximo passo

TODA SESSÃO TERMINA COM:

- Sumário dos decisões tomadas
- Lista de questões abertas
- Prévia do próximo artefato

CONTROLE DE VERSÃO DOS ARTEFATOS:

artefato\_01\_v1.0 → artefato\_01\_v1.1 → ...  
(versão muda quando requisito muda)

---

 *Artefato vivo. Versão: 1.0 | Sessão: #01 | Série: IA×Energia×Web3 Próximo artefato gerado na próxima sessão de trabalho.*